INDEX

NAME: P. Baktialakshmy ROLL NO: 240701067

SUBJECT: Computer Networks Observation STD: CSE DIV: AG

SCHOOL: Rajalakshmi Engineering College - Thandalam

SR.NO.	DATE	TITLE	PAGE NO.	SIGNATURE
1	20.07.26	Study of various commands used in linux & windows		28/8/26
2	28.07.26	Study of Network Cables		4/8/26
3	04.08.26	Cisco Packet Tracer		10/8/26
4	10.08.26	Wireless LAN		10/8/26
5	18.08.26	Wireshark		28/9/26
6	7.09.26	Error detection & correction using Hamming code.		22/9/26
7	22.09.26	Nmap to discover live host using TryHackme platforms		22/9/26
8	28.09.26	End to End communication at transport layer.		22/9/26
9	28.09.26	SLIDING WINDOW PROTOCOL		29/09/26
10	6.10.26	Subnetting in cisco packet tracer		6/10/26
11	6.10.26	ping program.		6/10/26

SR.NO.	DATE	TITLE	PAGE NO.	SIGNATURE
12	13/10/26	Using RAW SOCKETS to implement packet sniffing		
13	13/10/26	Internetworking using cisco packet tracer.		
14	13/10/26	Routing information protocol using cisco.		

Completed

20/07/22

Exp.1 Study of various Network Commands used in Linux and Windows.

Aim:-

Study of various Network commands used in linux and windows.

Basic Networking commands:-

arp -a :- ARP is short form of address resolution protocol, It will show the IP address of your computer along with the IP address and MAC addre of your router.

hostname:- This is the simplest of all TCP/IP commands. It simply displays the name of your computer.

For Ex:- Bakkia-Pc

Ping:- It sends a small message to another computer.

Ex:- ping google.com

Checks whether another computer or website is reachable.

Ipconfig:- Shows your computer network details.

Ex:- IP address, subnet mask, default Gateway.

IPv4 Address : 192.168.1.100

It tells you your computer's network address.

netstat:- Display active network connections & listening ports. Ex:- TCP 192.168.1.10:5000

nslookup:- Find the IPaddress of a website.

Ex:- google.com
142.250.xxx.xxx.

Converts website name into its IP address.

nbstat -a;- Displays NetBIOS information and helps troubleshoot NetBIOS name resolution.

Pathping:- Shows the route and packet loss statistics.

Ex:-
-i address use the specified source address.

route:- Displays or modifies the route table.

Ex:- -f, -p.

Linux Networking Commands:-

IP address show:- Display IP address assigned to the interface.

IP address add 192.168.1.251/24 dev enps03 :- Assigns an IP address to a network interface.

IP address del 192.168.1.254/24 dev enps03 :- Remove an IP address from an interface.

ip link set etho up:- Enables a network interface.

IP link set ethodown:- Disables a network interface.

nmcli connection show:- List the Network manager connection profiles.

ifconfig:- Displays or configures network interfaces.

mtr <hostname/IP>:- Combines the features of ping and

traceroute for network diagnostics.

ip route show:- display the IPv4 default gateway.

cat /etc/resolv.conf:- Display DNS of the server.

ip link set eth0 promisc on:- This command enables promiscuous mode on the eth0 interface. In this mode, the interface receives all network packets passing through the network, not just those addressed to it. It is commonly used for network monitoring and analysis.

ip route add default via 192.168.1.254 dev eth0:- This command adds a default gateway to the routing table. The default gateway 192.168.1.254. It tells the system how to reach that network.

ip route add 192.168.1.0/24 dev eth0:- This command adds a route to the 192.168.1.0/24 network using the eth0 network interface directly, without specifying a gateway.

ip route delete 192.168.1.0/24 via 192.168.1.254 This command removes the specified route from the routing table. It is useful when an existing route is no longer required.

ip route get 10.10.1.4:- This command display the route that the system will use to reach the destination IP address 10.10.1.4. It helps verify routing decisions.

Ping localhost :- Tells whether the local TCP/IP network stack is working correctly.

mtr google.com:- shows you the statistics, including each hop with time and loss %.

mtr -g google.com: shows IP address, if use -g you will get IP addresses instead of host names.

mtr -b google.com:: shows the numeric IP address and hostnames.

mtr -c 10 google.com!- set the number of Pings that you want to send.

dnf install -y tcpdump:- install the tcpdump

tcpdump -D: Before starting, you will need to know use sudo or have root access.

tcpdump -i eth0:- you can initiate that with tcpdump -i eth0 sample output.

tcpdump -i eth0 -c 10: capture traffic to and from one host.

tcpdump -i eth0 -c 10 host 8.8.8.8:- you can filter out traffic coming from a specific host.

tcpdump -i eth0 src host 8.8.8.8 :- For traffic coming from 8.8.8.8.

tcpdump -i eth0 dst host 8.8.8.8!- For outbound traffic going to 8.8.8.8.

tcpdump -i eth0 net 10.1.0.0 mask 255.255.255.0 capture traffic to and from a network.

tcpdump -i eth0 port 53:- Capture only DNS port 53 traffic.

tcpdump -i eth0 -c 10 host www.google.com and port 443:- To capture only HTTPS traffic

tcpdump -i eth0 port not 53 and not 25:- To capture all port except port 80 and 25

Result:—

Hence the various networking commands used in linux and windows are studied and executed successfully.

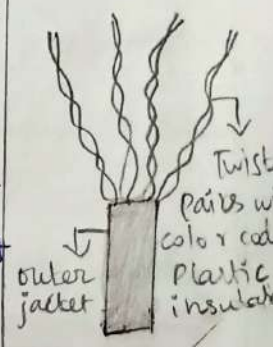
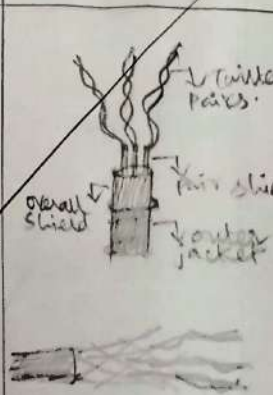
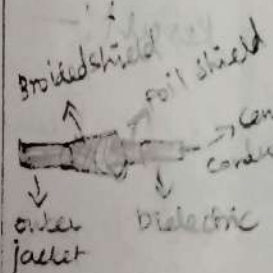
28/8/20

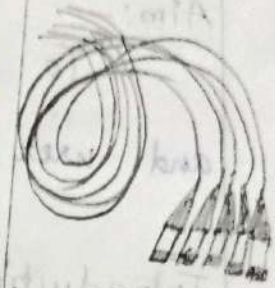
2nd/2nd Exp:2 Study of Network Cables.

Aim:- Study of different types of Network cables.

Different types of cables used in networking are:-

* Unshielded Twisted pair (UTP) cable.
* Shielded Twisted pair (STP) cable.
* Coaxial cable
* Fibre optic cable.

Cable type	category	Maximum data transmission	Advantages & Disadvantages	Application/use	Image
UTP	Category 3	10 bps	Adv:- * Cheaper in cost. * Easy to install as they have a smaller hole diameter. dis:- * More prone to (EMI) Electromagnetic Interference and noise.	10Base-T Ethernet .Fast Ethernet Gigabit Ethernet. Fast Ethernet Gigabit ethernet	
	Category 5	upto 100 Mbps			
	Category 5e	1Gbps			
STP SSTP	Category 6, 6a	10Gbps	Adv:- * Shielded. * Faster than UTP. * Less susceptible to noise and interference dis:- * Expensive * Greater installation effort	Gigabit Ethernet 10G Ethernet (55 m) widely used in data centres. Gigabit Ethernet 10G Ethernet (100 m)	
	Category 7	10Gbps			
Coaxial Cable	RG-6 RG-59 RG-11	10-100 Mbps	Adv:- * High bandwidth. * Immune to interference. * Low loss bandwidth. * Versatile dis:- * Limited distance. * cost. size is	Speed of signal is 500 m Television network high speed internet connections.	

fibre optics cable	single mode multi mode	100 Gbps	Adv:- * High speed * High bandwidth * High security * Long distance. Dis:- * Expensive * Requires skilled installers	maximum distance of fibre optics cable is around 100 meters.	

Result:-

The different types of network cables are learned and executed successfully....

4/8/21

04/08/26

EXP. 3 CISCO PACKET TRACER.

Aim:-

To study cisco packet Tracer tool Installation and user interface overview.

Introduction:-

A simulator, as the name suggests, simulate network devices and its environment. Packet Tracer is an excisting network design, simulation and modelling tool.

* It allows you to model complex systems without the need for dedicated equipment.
* It helps you to practice your network configuration and troubleshooting skills via computer or an Android or iOS based mobile device.
* It is available for both the Linux and Windows desktop environments.
* Protocols in Packet Tracer are coded to work and behave in the same way as they would on real hardware.

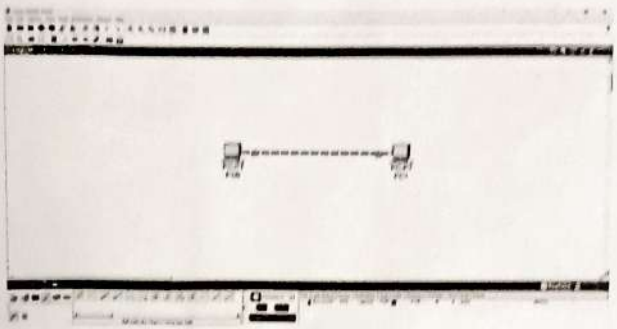
Analyze the behaviour of network devices using CISCO PACKET TRACER simulator.

i) Peer to peer
ii) Switch
iii) Hub.

i) Peer to Peer

A peer to peer network is a network in which computers communicate directly with each other without requiring a dedicated central server.

Ex:
Two PC's connected directly wing a cable and communicating with each other.

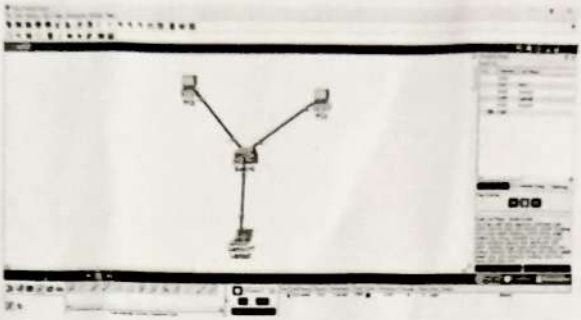


ii) switch

A switch is a networking device used to connect multiple devices in a network and forward data only to the intended destination device.

Ex:
PC1 -> switch -> PC3

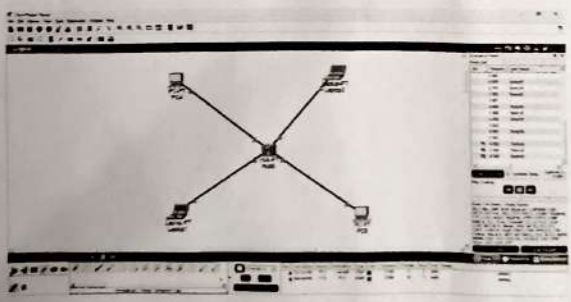
switch knows where PC3 is and forwards the frame only toward PC3. Switch only works at the Data Link layer (layer 2) of the OSI model. It uses MAC address. sends data to the required port.



iii) Hub

A Hub is a basic networking device used to connect multiple computers in a network. It receives data from one device and broadcasts it to all connected devices. Hub works mainly at physical layer (layer 1) of the OSI model.

Ex:- If PC1 sends data to PCs through a hub, the hub sends that data to PC2, PC3, PC4 etc.



Result:-

Cisco Packet Tracer are learned and executed successfully.

10/08/25

Exp:1 Setup and configure a LAN using switch and Ethernet cables

What is LAN?

LAN stands for Local Area Network which is a group of computers and connected devices linked together in a small physical location like a home, school, or office building.

How to set up LAN

Step 1 :- Connect the Hardware

→ Place your router in a central location near a power outlet.

→ Connect your modem to the router's WAN or Internet port using an Ethernet cable

→ Plug the router into power and wait for it to turn on.

Step 2 :- Connect your devices

→ Wired setup :- plug one end of an Ethernet cable into your computer's network port and the other end into one of the numbered LAN ports on the router.

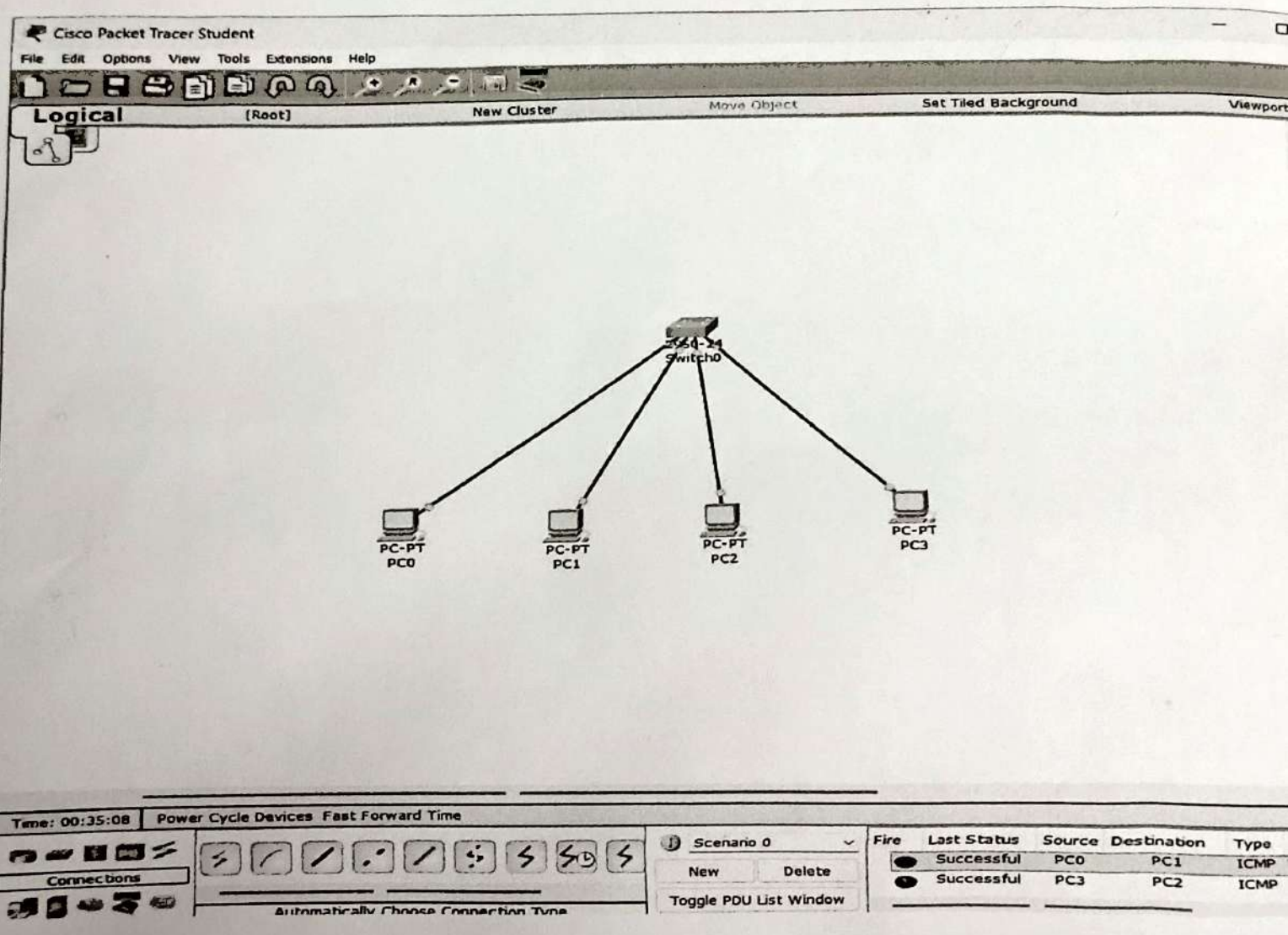
Step 3 :- Configure the router

→ Type your IP address (192.168.1.1 or: 192.168.0.1) into the address bar and Press Enter.

→ Login using the default username and password printed on the router.

Step 4:- Check the connectivity between switch and other machine by using ping command in the command prompt of the device.

Step 5:- Try to access the shared folder from others computers of the network



18/08/20

EXP:5 WIRESHARK

Aim:- Experiments on packet capture tool: Wireshark.

To filter, capture, view, packets in Wireshark Tool. Capture 100 packets from the Ethernet: IEEE 802.3 LAN Interface and save it.

Procedure:-

* Select Local Area Connection in Wireshark
* To capture option.
* select stop capture automatically after 100 Packets.
* Then click start capture.
* Search TCP packets in search bar.
* To see flow graph click statistics flow graph.
* Save the packets.

1. Create a Filter to display only TCP/UDP packets, inspects the packets and provide the flow graph.

* Select LAN connection in wiresharks.
* To capture option
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Save the packets.

2. Create a filter to display only ARP packets and inspect the packets.

Procedure:-

* Go to capture option.
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Search ARP packets in search bar.
* Save the packets.

3. Create a filter to display only HTTP packets and inspect the packets procedure

* Select local Area Connection in Wireshark
* Go to capture option.
* Select stop capture automatically after 100 packets.
* Then click start capture.
* Search HTTP packets in search bar.

4. Create a file to display only DNS packets and provide the flow graph.

Procedure:-

* Go to capture option.

* Select Stop capture automatically after 100 packets.

* Then click Start capture.

* Search DNS packets in search bar.

* To see the flow graph click statistics flow graph.

5. create a filter to display only DHCP packets and inspect the packets.

* Select local Area connection in Wireshark

* Go to capture option.

* Select stop capture automatically after 100 packets.

* Then click start capture.

* Search DHCP packets in search bar.

* Save the packets.

Result:-

Wireshark tool on packet capture tool experiment are learned and executed successfully.

28/9/20

7/09/26

Exp. 6 Error detection & correction using Hamming code.

Aim:-

Write a program to implement error detection and correction using hamming code concept. Make a test run to input data stream and verify error correction feature.

Sender:-

* Input to sender file should be a text of any length. Program should be convert the text to binary.

* Apply hamming code concept on the binary data and add redundant bits to it.

* Save this output in a file called channel.

Code:-

```
data = input("Enter 4 bits:")
d1, d2, d3, d4 = map(int, data)
P1 = d1 ^ d2 ^ d4.
P2 = d1 ^ d3 ^ d4
P4 = d2 ^ d3 ^ d4
Code = [P1, P2, d1, P4, d2, d3, d4]
print("Sender Hamming Code:", "".join
    (map(str, code))).
```

Receiver side :-

* Receive the hamming code from the sender.
* Enter the received code.
* Calculate the parity check values.
* Find the error position using the parity values.
* Correct the error by changing the bit.
* Display the corrected hamming code.

Code:-

```
r = list(map(int, input("Enter received 7 bits:")))
r1 = r[0] ^ r[2] ^ r[4] ^ r[6]
r2 = r[1] ^ r[2] ^ r[5] ^ r[6]
r3 = r[3] ^ r[4] ^ r[5] ^ r[6]

error = r1 * 1 + r2 * 2 + r4 * 4

if error == 0:
    print("NO Error")
else:
    print("Error at position:", error)
    r[error-1] ^= 1
    print("corrected code:", "".join(map(
        str, received )))
```

Input:-

Enter 4 data bits : 1011

Sender Hamming code : 0110011

Received 7 bits : 0010011

Output:

Error at position : 3

corrected code : 0110011

Sample Input 2 :-

Enter 4 data bits : 0010

Sender Hamming Code : 0101010

Enter received 7 bits : 0101010

Output :-

NO Error

Result :-

Error detection & Error correction using hamming code was executed successfully.

28/9/23

22/09/26

Exp.7 NMAP to Discover Live Hosts Using Nmap Scans on the TryHackme Platform.

Aim:-

To Discover Live Hosts using Nmap Scans (ARP, ICMP, TCP/UDP) on the TryHackMe Platform Room Link: https://tryhackme.com/room/nmapol.

Procedure:-

Task 1:- Introduction

* ARP Scan: This scan uses ARP requests to discover live hosts.

* ICMP scan: This scan uses ICMP requests to identify live hosts.

* TCP/UDP Ping scan: This scan sends packets to TCP ports and UDP ports to determine live hosts.

Task 2:- Subnetworks

Network segment - It is a collection of computers linked together through a common "medium".

Subnetworks - It is usually the equivalent of one or more networks segments linked together and configured to utilize the same router in an IP network.

* Subnets with /16
* Subnets with /24

Task 3 :- Enumerating Targets

We need to identify the targets we want to scan before we conduct any "action"

1) List
- <MACHINE - IP> <scanme.nmap.org> <example.com>
- will scan 3 IP addresses

2) range
- 10.11.12.15-20
- will scan 6 IP addresses

3) Subnet
- <MACHINE - IP> /30
- will scan 4 IP addresses

nmap -SL <TARGETS>

Task 4 :- Discovering Live Hosts

1) ARP, from Link layer - commonly used for Man-In-Middle-Attack (ARP spoofing and ARP poisoning).

2) ICMP from Network layer - commonly used in DDOS attacks.

3) TCP from Transport layer - SYN flooding, TCP reset, TCP session hijacking attacks.

4) UDP from Transport layer - Also used in DDOS attacks.

Task 5 :- Nmap Host Discovery using ARP

1) When a privileged user tries to scan LAN (Local area Network), Nmap uses ARP Requests.

2) to discover internet hosts without port scanning - live systems use nmap -sn Targets.

3) nmap -PR -sn <TARGETS>
-PR specifies that you only want an ARP scan.

4) nmap -PR -sn <MACHINE IP>/24

5) nmap -PR -sn <ATTACKBOX IP>/24

Task 6 :- Nmap host discovery using ICMP

1) Add -PE option to utilize ICMP echo request to discover live hosts.

2) nmap -PE -sn TARGET to check host is live.

3) -PP option instructs Nmap to use ICMP timestamp requests.
nmap -PP -sn TARGET

4) -PM option can enable this scan to ICMP address mask requests.
nmap -PM -sn TARGET

Task 7 :- Nmap Host discovery using TCP and UDP.

1) Nmap to use TCP SYN Ping use -PS followed by the port number, range, list or a combination of this options.
-PS21, -PS25 targets ports 21, 22, 23, 24, 25.

2) nmap -PS -sn TARGET Check the TCP port is open.

3) TCP ACK ping - you must be running Nmap as a privileged user.

4) nmap -PA -sn TARGET to discover live online hosts on the target's subnet.

5) UDP ping - sending a UDP packet to an open port is unlikely to result in a response

6) If we transmit a UDP packet to a closed UDP port, we should expect to receive an ICMP port unreachable packet.

nmap -PU -sn TARGET.

Task 8:- Using Reverse-DNS Lookup

1) Nmap is to use reverse-DNS interrogate hosts.

2) -R option to query the DNS server even for offline hosts.

Task 9:- Summary

ARP scan, ICMP Echo scan, ICMP Timestamp scan, ICMP address mask scan, TCP SYN ping scan, TCP ACK ping scan, UDP ping scan.

Result:-

NMAP to discover live host using tryhackme platform was successfully executed.

Exp. 8 End to End Communication at Transport layer.

a)

Aim:- Implement echo client server using TCP/ UDP Packets.

Procedure:-

* Create a socket using Python socket programming.
* Specify the IP address and port number
* Bind the server socket to the IP address and port.
* Make the server listen for client connection
* Connect the client to the server.
* Send and receive messages between client and server.
* Display the received message.
* Close the socket connection.

Code:-

TCP - Server

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind("127.0.0.1", 5000))
server.listen(1)
print("Server is waiting for connection...")
con.n, addr = server.accept()
Print ("Client connected")
```

```
while True:
    data = conn.recv(1024).decode()
    if not data:
        break
    print("client:", data)
    conn.send(data.encode())
conn.close()
server.close()
```

TCP_Receiver (Client)

```
import socket
client = socket.socket(socket.AF_INET,
Socket.SOCK_STREAM)
client.connect(("127.0.0.1", 5000))
message = input("Enter message:")
client.send(message.encode())
reply = client.recv(1024).decode()
print("server:", reply)
client.close()
```

Terminal 1 - server

```
python tcp_server.py
server is waiting for
connection.....
client connected
Client: Hello.
```

Terminal 2 - Client

```
python tcp_client.py
Enter message: Hello
Server: Hello
```

TCP echo is working.

Result:-

The TCP/UDP packets using echo client server socket program is executed successfully.

b)

Aim:- Implement chat client server using Tcp packets.

Code:-

Chat - server side

```
import socket
server = socket.socket(socket.AF_INET, socket.
SOCKET_STREAM)
server.bind(("127.0.0.1", 6000))
server.listen(1)
print("waiting for client....")
conn, addr = server.accept()
print("client connected")
while True:
    message = conn.recv(1024).decode()
    if message.lower() == "bye":
        print("client ended chat")
        break
    print("client:", message)
    reply = input("server:")
    conn.send(reply.encode())
    if reply.lower() == "bye":
conn.close()
server.close()
```

Chat - client side

```
import socket
client = socket.socket(socket.AF_INET, socket.
```

```
SOCK_STREAM)
client.connect(("127.0.0.1", 6000))
print("connected to server")
while True:
    message = input("client: ")
    client.send(message.encode())
    if message.lower() == "bye":
        break
    reply = client.recv(1024).decode()
    print("server:", reply)
    if reply.lower() == "bye":
        break
client.close()
```

Terminal 1

```
python chat_server.py
waiting for client.....
client connected
client: Hello
Server: Hi
client: How are you?
Server: Fine
```

Terminal - 2

```
python chat_clie
connected to se
Client: Hello
server: Hi
client: How are yo
server: Fine.
```

Result:-

The Chat programming using sockets by TCP client & server was implemented successfully.

28/09/26

EXP 9. SLIDING WINDOW PROTOCOL

Aim:-

Write a program to implement flow control at data link layer using sliding window protocol. simulate the flows from one node to another.

Code:-

```
import time
window = int (input ("Enter window size: "))
message = input ("Enter message: ")
start = 0
while start < len(message)
print ("SENDER")
end = start + window
if end > len (message):
    end = len (message).
#Sender buffer
sender_buffer = [ ]
for i in range (start, end):
    frame = (i, message [i])
    sender_buffer.append (frame)
    print ("Frame NO:", i, "DATA:", message[i])
Print ("\n Sender Buffer: ")
print ( sender_buffer)
# Delay
  print ("\n Waiting for receiver...")
  time. sleep(1)
#Receiver
  Print ("RECEIVER")
  expected = start
  error = False
for frame in sender_buffer:
```

```
frame_no = frame[0]
data = frame[1]
print("Received Frame:", frame_no, "DATA:", da
if frame_no == expected:
    expected = expected + 1
else:
    error = True
    break

#Receiver buffer
if error:
    receiver_buffer = ("NACK", expected)
    print("\n Receiver Buffer:")
    print(receiver_buffer)
else:
    ack = expected - 1
    receiver_buffer = ("ACK", ack)
    print("\n Receiver buffer:")
    print("receiver_buffer")
# sender reads ACK/NACK
print("\n SENDER CHECKING RESPONSE")
response = receiver_buffer[0]
number = receiver_buffer[1]
if response == "ACK":
    print("ACK received :", number)
    start = number+1
else:
    print("NACK received:", number)
    start = number
print("TRANSMISSION COMPLETED").
```

Output:

Enter window size: 3

Enter message: Hello

SENDER

Frame NO: 0 DATA: H

Frame NO: 1 DATA: e

Frame NO: 2 DATA: l

Sender Buffer:

[(0,'H'), (1,'e'), (2,'l')] waiting for receiver...

RECEIVER

Received Frame: 0 DATA: H

Received Frame: 1 DATA: e

Received Frame: 2 DATA: l

Receiver Buffer:

('ACK', 2)

SENDER CHECKING RESPONSE

ACK Received: 2

SENDER

Frame NO: 3 DATA: l

Frame NO: 4 Data: 0

Sender Buffer:

[(3,'l'), (4,'0')]

waiting for receiver...

RECEIVER

Received frame: 3 DATA: l

Received Frame: 4 DATA: 0

Receiver Buffer:

('ACK', 4)

SENDER CHECKING RESPONSE

ACK Received: 4

TRANSMISSION COMPLETED.

Result:-

Sliding window protocols are learned and executed successfully.

29/9/20

6/10/25

EXP 10 Subnetting in Cisco Packet Tracer

Aim:-

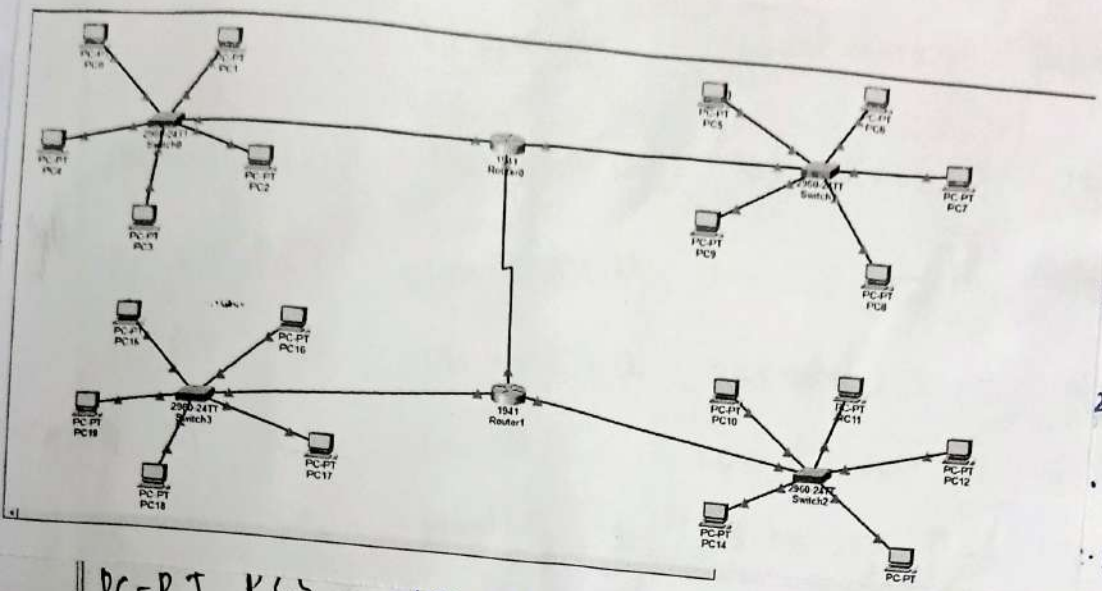
To implement classless IP subnetting using Cisco packet Tracer and verify the connectivity between different subnets.

Requirements:-

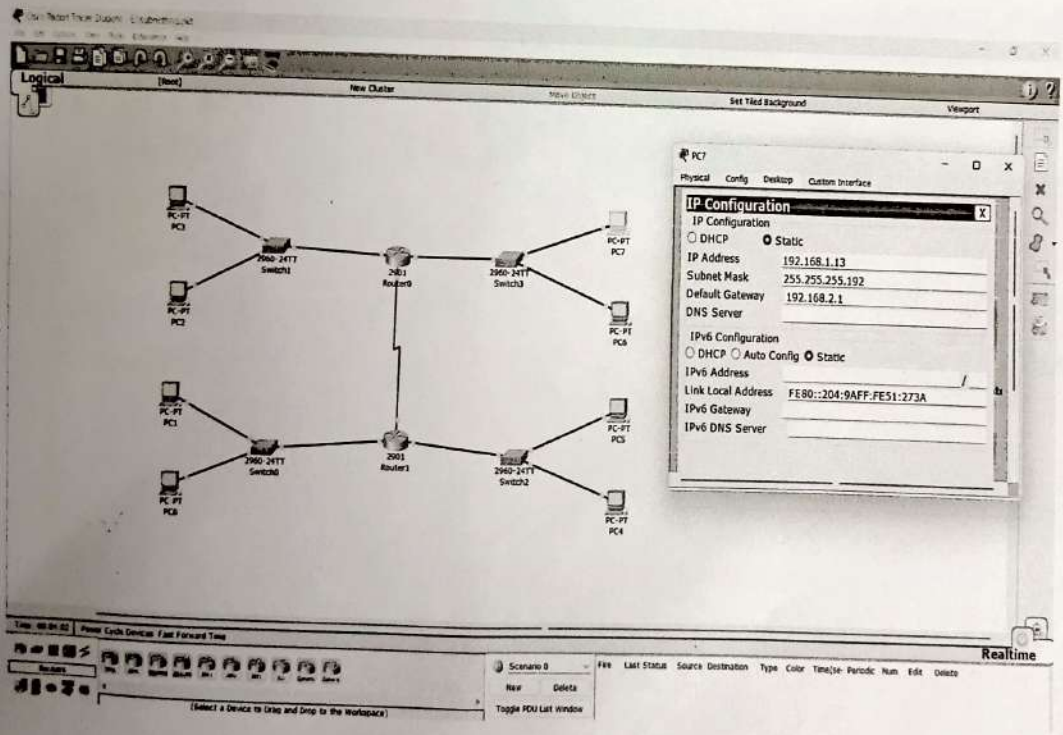
* Cisco packet Tracer
* Routers
* switches
* PCs
* Ethernet cables.

Procedure:-

1. Open Cisco packet Tracer
2. Create a new network topology.
3. Add routers, switches, and PCs to the workspace. connect the device using ethernet cables.
4. Divide the network into smaller subnets using the /27 subnet mask.
5. Configure the IP address and subnet mask of the router interfaces.
6. Configure the IP address, subnet mask and default gateway for each PC.
7. Configure the required switch ports.
8. Enable the router interfaces using the no shutdown command.
9. Check the IP configuration of all devices. Use the ping command to check connectivity between PCs.
10. Verify that the devices in the network communicate



PC-PT PC5	192.100 ...		
PC-PT PC6	192.168.1.14	255.255.255.192	192.168.2.1
PC-PT PC7	192.168.1.13	255.255.255.192	192.168.2.1



Result:-

Thus, Classless IP subnetting was successfully implemented using Cisco Packet Tracer, and connectivity between the network devices was verified successfully.

6/10/25

EXP 11 PING PROGRAM

Aim:-

To implement a ping program using socket programming to check the connectivity between two network devices.

Program:-

```
import socket
import time
import sys
def udp_ping_server(host="127.0.0.1", port=12000):
    server_socket = socket.socket(socket.AF_INET,
                                  SOCK_DGRAM)
    server_socket.bind((host, port))
    print(f"[Server] Listening on {host}:{port}")
    while True:
        msg, client_address = server_socket.recvfrom
                                  (1024)
        print(f"[SERVER] Received '{msg.decode()}
            from {client_address}")
        server_socket.sendto(msg, client_address)
def udp_ping_client(server_host="127.0.0.1", server_port=
12000, count=5):
    client_socket.settimeout(1)
    for i in range(1, count+1):
        msg = f"Ping {i} {time.time()}"
        start = time.time()
        client_socket.sendto()
            msg.encode(), (server_host, server_
        port)
```

```
try:
    data, _ = client_socket.recvfrom(1024)
    end = time.time()
    rtt = (end - start) * 1000
    print(
        f"Reply from {server_host}:{server_port} |
    "f"{data.decode()} | RTT = {rtt:.2f}ms")

except socket.timeout:
    print(f"Request {i} timed out")

client_socket.close()

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "server":
        udp_ping_server()
    else:
        udp_ping_client()
```

Output:-

```
python udp_ping.py server
[SERVER] Listening on 127.0.0.1:12000
[SERVER] Received 'ping 1 1791476483.6564088' from
  ('127.0.0.1', 58522)
[SERVER] Received 'ping 2 1791476483.6980255' from
  ('127.0.0.1', 58522)
python udp_ping.py
  Reply from 127.0.0.1:12000 | Ping 1 1791476483.6564088 | RTT
= 0.92ms.
  Reply from 127.0.0.1:12000 | Ping 2 1791476483.6980255 | RTT
= 0.25ms
```

Result:-

Thus, the Ping Program is successfully executed.

6 | 10 | 25

13/10/26

EXP 12 USING RAW SOCKETS to implemen Packet sniffing.

Aim:-

To write and execute a Python Program u RAW Sockets to capture and analyze network packets.

Program:-

```
from scapy.all import sniff
from scapy.layers.inet import IP, TCP, UDP, IC
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst
        protocol_name = ""
        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protol_name = "Unknown Protocol"
        print(f"Protocol: {protocol_name}")
        print(f"Source IP: {src_ip}")
        print(f"Destination IP: {dst_ip}")
        print("_" * 50)
```

filter = "ip", store = 0)

Output:-

Protocol : TCP
Source IP : 192.168.1.6
Destination IP : 57.155.141.117

Protocol : TCP
Source IP : 192.168.1.6
Destination IP : 192.168.1.1

Protocol : TCP
Source IP : 57.155.141.117
Destination IP : 192.168.1.6

Result:-

Thus, the program was successfully executed.

14/04/26

13/10/25 Exp. 13 Internetworking with routers in CISCO PACKET TRACER

Aim:-

To design and configure a simple internetwork using a router in cisco packet tracer and fast connectivity between two PC's

Procedure:-

Step 1:- Configuring Router 1

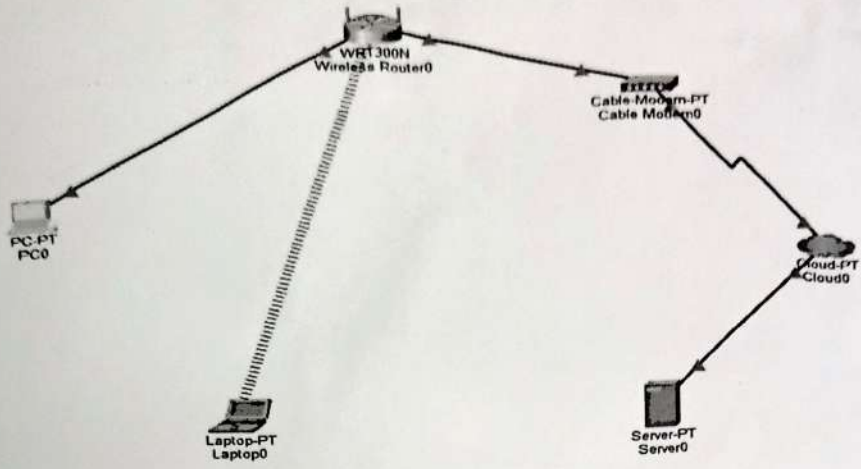
* Select the router and open CLI.
* Press enter to start configure router1
* Type enable to activate the previleged mode.

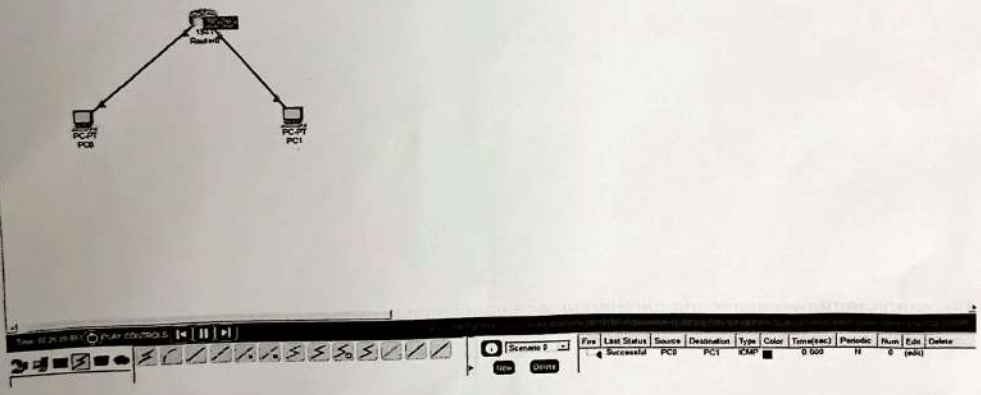
Step 2:-

* Assign IP address to every PC in the network
* Select the PC, Go to the desktop and select IP configuration and assign an IP addr default gateway, subnet mask.
* Assign defaut gateway for both PC's.

Step 3:-

* Connect Fast ethernet for both PC's through copper straight-through cable.
* Connect fast ethernet router using a copper straight through cable





Result!-

Thus, the simple internetworking with routers is succesfully designed and configured.

4/10/'24

13/10/20

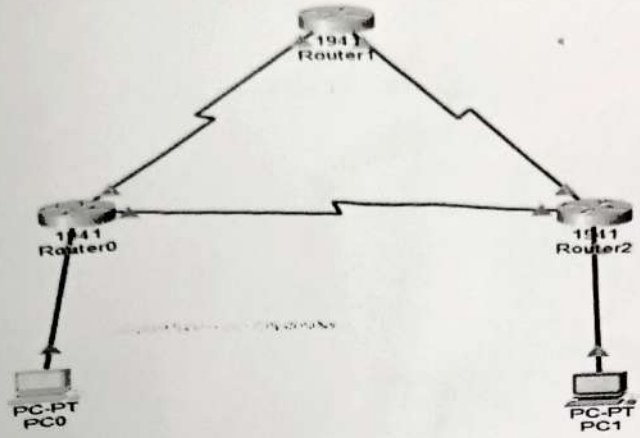
Exp 14 Routing Information Protocol

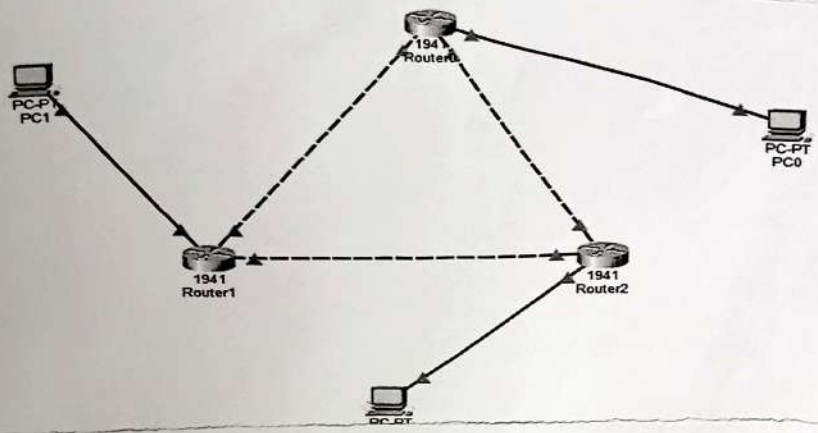
Aim:-

To simulate a static routing configuration using cisco packet tracer.

Procedure:-

* open cisco packet tracer and create the given network topology.
* Connect 3 routers and PC's using suitable cables.
* Configure IP addresses for routers and PC's.
* Enable all router interface using no shutdown.
* Configure static routers on router 0, router 1 and router 2 using IP mobile
* Set main and backup router using different administrative distance value.
* Test connectivity.
* Use tracer to verify the routing path
* Thus a static routing is successfully configured and verified.






```
Pinging 20.0.0.2 with 32 bytes of data:

Reply from 20.0.0.2: bytes=32 time=55ms TTL=125
Reply from 20.0.0.2: bytes=32 time=56ms TTL=125
Reply from 20.0.0.2: bytes=32 time=60ms TTL=125
Reply from 20.0.0.2: bytes=32 time=55ms TTL=125

Ping statistics for 20.0.0.2:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 55ms, Maximum = 60ms, Average = 56ms

C:\>tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2   11 ms      8 ms      5 ms      192.168.1.250
  3   29 ms     26 ms     25 ms      192.168.1.245
  4   22 ms     28 ms     20 ms      20.0.0.2

Trace complete.
```

```
Router>enable
Router#show ip route static
S    30.0.0.0/8 [20/0] via 40.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2

Router#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

Router#show ip route static
S    30.0.0.0/8 [10/0] via 20.0.0.2
S    50.0.0.0/8 [10/0] via 40.0.0.2
S    60.0.0.0/8 [1/0] via 40.0.0.2
```

RIP successfully configured and dynamic routing between PC0 and PC1 was verified.

14/10/25